

COBOL — Reference

COBOL here is a free-format subset compiled with `lex` + `yacc` + `cc` (`cobol.l` / `cobol.y` lower COBOL to C; `cc` lowers the C to .NET IL). The four divisions map cleanly: **PROGRAM-ID** names it, **WORKING-STORAGE** builds the data, **PROCEDURE DIVISION** becomes the code. Each **paragraph** is emitted as a `public static` method on `CProgram`, so C#/VB.NET can invoke compiled COBOL.

Free-format (vs the columnar fixed format) is the escape from COBOL's column pathology: columns don't matter and `*>` starts a comment. Two COBOL-specific scanner problems are handled without lexer start-states: the **PICTURE clause** is grabbed as a single token (an internal `.` is kept only when glued to a following picture char, so the sentence-ending period survives), and a lone `.` is the statement **terminator**.

```
cobol prog.cob           # compile to prog.exe (native .NET executable)
cobol prog.cob -o app.exe # choose the output name
cobol lib.cob --dll       # compile to a library for C#/VB.NET to reference
```

Quick Reference

```
IDENTIFICATION DIVISION.  PROGRAM-ID. NAME.
DATA DIVISION.            WORKING-STORAGE SECTION.
01 COUNTER  PIC 9(4) VALUE 0.           *> numeric (4 digits)
01 NAME     PIC X(20) VALUE "Ada".      *> text
01 AMOUNT   PIC 9(5)V99.                *> implied decimal point
01 PRETTY   PIC ZZ,ZZ9.99.              *> edited (zero-suppress, comma, point)
01 TABLE-X PIC 9(3) OCCURS 10.          *> array, subscript TABLE-X(i), 1-based
01 AGE      PIC 9(3) VALUE 30.
      88 IS-ADULT VALUE 18 THRU 120.    *> condition-name (a boolean test)
PROCEDURE DIVISION.
    DISPLAY "text" FIELD.      MOVE x TO y.      ACCEPT y.
    ADD a b TO c [GIVING d].   SUBTRACT a FROM b. MULTIPLY a BY b. DIVIDE a INTO b.
    COMPUTE d = (a + b) * c.
    IF cond THEN ... ELSE ... END-IF.
    EVALUATE n WHEN 1 ... WHEN 2 ... WHEN OTHER ... END-EVALUATE.
    PERFORM PARA.             PERFORM PARA 5 TIMES. PERFORM PARA UNTIL cond.
    PERFORM VARYING i FROM 1 BY 1 UNTIL i > 10 ... END-PERFORM.
    STOP RUN.
```

Data types

Every item has a **PICTURE** that gives its category and size:

- **9** numeric digit — `PIC 9(4)` is a 4-digit integer; `PIC 9(5)V99` has an implied decimal point (a fixed-point number).
- **X** any character, **A** letter — `PIC X(20)` is a 20-character string.
- **edited** (**Z** zero-suppress, **.** insertion, **\$**, sign) — a display field a number is formatted into on `MOVE`.
- **S** marks a signed number; **V** an implied decimal point.

`OCCURS n` makes an item an array (subscripted `T(i)`, 1-based). `VALUE` gives an initial value. An **88-level** under an item is a *condition-name* — a named boolean.

Statements / Commands

`DISPLAY`, `ACCEPT`, `MOVE`, the arithmetic verbs (`ADD`, `SUBTRACT`, `MULTIPLY`, `DIVIDE`, `COMPUTE`), `IF/ELSE/END-IF`, `EVALUATE`, `PERFORM` (out-of-line, `N TIMES`, `UNTIL`, `VARYING`, and inline `... END-PERFORM`), `GO TO`, and `STOP RUN`. A sentence is one or more statements ended by a period; block statements use scope terminators (`END-IF`, `END-PERFORM`, `END-EVALUATE`).

Functions

COBOL's unit of code is the **paragraph**, not the function. A paragraph is a name followed by sentences; `PERFORM` runs it (and returns). Each paragraph compiles to a `public static void` method `pg_<name>` on `CProgram`, which is what makes a compiled COBOL program callable from C#/VB.NET (Activity 9).

Input / Output

`DISPLAY a b c` writes the operands (no separators) and a newline; numeric items print per their PICTURE (zero-padded), edited items print their formatted text. `ACCEPT x` reads a line/number into `x`. (Console I/O only; file I/O — `SELECT / FD / READ / WRITE` — is a subset boundary.)

Graphics

None. COBOL's "picture" is the **PICTURE clause**: Activity 8 below uses edited pictures to format a money report — COBOL's native idea of drawing.

Notes

- **Free-format only.** `*>` comments; columns don't matter. A period ends a sentence.
- **Keywords are reserved** (a large COBOL reserved-word set).
- The PICTURE clause is captured as one token; `(n)` repeat-counts and an internal `.` are supported.
- Numeric items are 32-bit integers (`9`, `S9`); `V`-decimals and edited fields are computed in double precision for display.
- Paragraphs are emitted as functions and `main` calls them in source order, so a `STOP RUN` that halts before fall-through behaves like real COBOL.

Subset boundaries

A substantial core, not the whole language. Not included: fixed (columnar) format; the ENVIRONMENT DIVISION beyond a header; file I/O (`SELECT` / `FD` / `READ` / `WRITE`); the report writer; `STRING` / `UNSTRING` ; `INSPECT` ; reference modification (`x(1:3)`); `COPY` copybooks; `REDEFINES` ; group-level moves; called sub-programs with a `LINKAGE SECTION` / `PROCEDURE DIVISION USING` (so interop drives parameterless paragraphs); and `PERFORM THRU` .

Tutorial

Every example was compiled and run with `cobol` ; the output shown is real.

1. Your first program

```
IDENTIFICATION DIVISION.
PROGRAM-ID. HELLO.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NAME PIC X(5) VALUE "World".
01 WS-COUNT PIC 9(3) VALUE 0.
PROCEDURE DIVISION.
    DISPLAY "Hello, " WS-NAME.
    MOVE 5 TO WS-COUNT.
    ADD 10 TO WS-COUNT.
    DISPLAY "Count is " WS-COUNT.
    STOP RUN.
```

```
Hello, World
Count is 015
```

Note `015` — a `PIC 9(3)` item always displays its full three digits, zero-padded. That PICTURE-driven formatting is COBOL's signature.

2. Variables and data types

Data lives in WORKING-STORAGE, each item introduced by a **level number** and a **PICTURE**:

```
01 CUST-NAME PIC X(20).      *> 20-character text
01 BALANCE   PIC 9(6)V99.    *> number with two implied decimals
01 COUNTER   PIC 9(4) VALUE 0.
```

`9` is a digit, `X` a character, `V` an implied decimal point, `VALUE` an initial value. The PICTURE fixes both the *type* and the *width*.

3. Flow control

Block **IF** , multi-way **EVALUATE** , and the **PERFORM** loops:

```
EVALUATE GRADE
  WHEN 1 DISPLAY "grade one"
  WHEN 2 DISPLAY "grade two"
  WHEN OTHER DISPLAY "other grade"
END-EVALUATE.
IF TOTAL > 10 THEN
  DISPLAY "big"
ELSE
  DISPLAY "small"
END-IF.
```

With **GRADE = 2** and **TOTAL = 15** :

```
grade two
big
```

EVALUATE is COBOL's switch; block statements close with scope terminators (**END-IF** / **END-EVALUATE**) rather than a period.

4. Arrays

OCCURS makes a table; subscripts are 1-based and written with parentheses:

```
01 NUMS  PIC 9(2) OCCURS 5.
01 I      PIC 9(2) VALUE 0.
01 TOTAL PIC 9(4) VALUE 0.
...
PERFORM VARYING I FROM 1 BY 1 UNTIL I > 5
  MOVE I TO NUMS(I)
END-PERFORM.
PERFORM VARYING I FROM 1 BY 1 UNTIL I > 5
  ADD NUMS(I) TO TOTAL
END-PERFORM.
DISPLAY "Sum of 1..5 is " TOTAL.
```

```
Sum of 1..5 is 0015
```

NUMS(I) indexes the table; the inline **PERFORM VARYING** is COBOL's counted loop.

5. Subroutines and functions — paragraphs and PERFORM

COBOL structures code into **paragraphs**, invoked with **PERFORM**:

```
PROCEDURE DIVISION.  
MAIN-PARA.  
    PERFORM ADD-PARA VARYING I FROM 1 BY 1 UNTIL I > 5.  
    DISPLAY "Total is " TOTAL.  
    STOP RUN.  
ADD-PARA.  
    ADD I TO TOTAL.
```

```
Total is 0015
```

PERFORM ADD-PARA VARYING ... runs **ADD-PARA** once per value of **I**. **MAIN-PARA** ends in **STOP RUN**, so control never falls into **ADD-PARA** on its own — exactly COBOL's fall-through-unless-halted model.

6. Memory management

Storage is **static** — there is nothing to allocate or free:

- Every WORKING-STORAGE item is a fixed-size field that exists for the whole run.
- **OCCURS n** reserves **n** fixed slots; **PIC X(n)** reserves **n** characters.
- **VALUE** sets the initial contents once at start-up.

No heap, no pointers (**REDEFINES**, dynamic tables, and **LINKAGE** are subset boundaries).

7. Strings, condition-names, and editing

Text is **PIC X(n)**; **MOVE** copies with space-padding/truncation to the field width. An **88-level** gives a condition a name:

```
01 AGE PIC 9(3) VALUE 25.  
88 IS-ADULT VALUE 18 THRU 120.  
...  
IF IS-ADULT  
    DISPLAY "adult"  
END-IF.
```

```
adult
```

IS-ADULT reads as a boolean but is really the test **AGE ≥ 18 AND AGE ≤ 120** — COBOL's way of naming a condition.

8. Drawing a picture — a formatted report

COBOL's "picture" is the **PICTURE clause**; its drawing is a *formatted report*. An edited field (`ZZ,ZZ9.99`) turns a raw number into aligned, comma-grouped money:

```
01 A      PIC 9(6)V99 VALUE 1234.50.
01 B      PIC 9(6)V99 VALUE 99.99.
01 C      PIC 9(6)V99 VALUE 12000.00.
01 GRAND  PIC 9(7)V99 VALUE 0.
01 P      PIC ZZ,ZZ9.99.
PROCEDURE DIVISION.
    MOVE A TO P.
    DISPLAY "Item 1: $" P.
    MOVE B TO P.
    DISPLAY "Item 2: $" P.
    MOVE C TO P.
    DISPLAY "Item 3: $" P.
    COMPUTE GRAND = A + B + C.
    MOVE GRAND TO P.
    DISPLAY "Total : $" P.
    STOP RUN.
```

```
Item 1: $ 1,234.50
Item 2: $    99.99
Item 3: $12,000.00
Total  : $13,334.49
```

The `Z`s suppress leading zeros to spaces, the comma appears only once digits start, and the decimals line up — the report "draws" itself through the PICTURE.

9. Calling COBOL from C# / VB.NET

Compile a COBOL file as a library; each paragraph becomes a `public static` method on `CProgram` :

```
IDENTIFICATION DIVISION.
PROGRAM-ID. COBLIB.
PROCEDURE DIVISION.
SHOW-BANNER.
    DISPLAY "≡≡≡ Daily Report (compiled from COBOL) ≡≡≡".
SHOW-FOOTER.
    DISPLAY "--- end of report ---".
```

```
cobol lib.cob --dll          # → coblib.dll
```

A C# program that references `coblib.dll` (and `CRuntime.dll`) calls the paragraphs directly:

```
CProgram.pg_SHOW_BANNER();  
Console.WriteLine("    ... (C# does its own work here) ... ");  
CProgram.pg_SHOW_FOOTER();
```

```
≡≡≡ Daily Report (compiled from COBOL) ≡≡≡  
    ... (C# does its own work here) ...  
--- end of report ---
```

The COBOL runs as ordinary .NET methods interleaved with C#. Passing arguments in and out (a **LINKAGE SECTION** with **PROCEDURE DIVISION USING**) is the natural next step (*Subset boundaries*), along with file I/O and **STRING** / **UNSTRING** for fuller COBOL.